How The Bash Program Works?

השקפים המסומנים ב- מהווים חומר הרחבה.

התוכן המובא בשקפים אלו <u>אינו</u> מהווה חלק מהחומר המחייב לבחינה בקורס.

כיצד עובדת תכנית ה- bash?

- ה- bash עובד בלולאה:
  1. הצגת prompt
  2. קריאת פקודה [= ה- command line]
  3. ניתוח הפקודה
  4. עיבוד הפקודה
  5. ביצוע הפקודה

מבני נתונים שמנהלת תכנית ה- bash

- לצורך ביצוע עבודתו, תוכנית ה- bash מנהלת מספר מבני נתונים:
  - רשימת הפקודות האחרונות שבוצעו.
  - רשימת המשתנים
  -רשימת ה alias

- **ניתוח הפקודה**: הפקודה מופרדת למילים. התווים רווח ו- Tab משמשים כמפרידים בין מילים

- **עיבוד הפקודה**: ביצוע החלפות לקבלת פקודה פשוטה. מבוצעות החלפות היסטוריה, משתנים וכדומה

- **ביצוע הפקודה**: הפקודה הפשוטה שהתקבלה מבוצעת

עיבוד הפקודה {ללא IO redirection וכן ללא pipeline}

1. אם הפקודה מכילה שימוש בפקודות קודמות (ע"י השימוש בתו '!'), אזי תכנית ה- bash מבצעת History Substitution.

2. שורת הפקודה שהתקבלה נשמרת ברשימת ההיסטוריה של הפקודות שבוצעו.
במידה ובוצעו החלפות ההיסטוריה, אזי שורת הפקודה (לאחר ההחלפות) מודפסת למסך

```
> echo AAA
AAA
> echo !!
echo echo AAA
echo AAA
```

3. אם המילה הראשונה בפקודה היא "alias", אזי מתבצע alias substitution.

4. בצוע variable substitution.

5. בצוע File name substitution.

6. בצוע command substitution. המחלפת בתוצאת הפקודה back-quoted string (`... `)

ביצוע הפקודה

- לאחר בצוע כל ההחלפות הפקודה מבוצעת.
- ישנן שני סוגים של פקודות:

1. פקודות חיצוניות
   - פקודה שה- bash 'לא יודע' לבצע 'בעצמו'
   - כלומר, הוא 'זקוק' לתוכנית אחרת [= 'חיצונית'] שתבצע את הפקודה שהמשתמש מבקש לבצע
   - 'תכנית אחרת' = תכנית ששמורה בקובץ executable או קובץ script
2. פקודות פנימיות
   - פקודה שה- bash יודע' לבצע 'בעצמו'
   - כלומר, הקוד של ה- bash 'מכיל' קוד {פונקציה} לביצוע הפקודה שהמשתמש מבקש לבצע

ביצוע הפקודה

תכנית ה- bash בודקת את שם הפקודה [= הארגומנט הראשון {= argv[1]} בשורת הפקודה, כדי לקבוע האם מדובר בפקודה חיצונית או פנימית.

פקודות חיצוניות

- ניתן להריץ <u>כל תוכנית</u> הקיימת במערכת [= השמורה בקובץ executable במערכת הקבצים]
- לצורך כך:

1. ה- bash יצור תהליך חדש. תהליך בן.
   - על ידי קריאה ל- fork()
   - בשלב הזה: תהליך הבן מהווה העתק של תהליך האב [= של תהליך ה- bash]
2. 'שינוי' משימת העבודה [= הקוד + הדאטה] ה'משויכת' לתהליך הבן.
   - על ידי קריאה לאחת מהרוטינות exec()
   - הארגומנט הראשון המועבר ל- exec() הוא path name של קובץ executable
     - ה- path name הזה אמור להיות {!!} ה- path name של קובץ executable
     - אשר ביצועו [= הרצתו] 'יגרום' לביצוע הפקודה שהמשתמש מבקש לבצע.

- הארגומנטים הנוספים המועברים ל- exec() הם ה- command line arguments

פקודות חיצוניות

- בכדי לבצע path resolution [= 'למצוא' את קובץ ה- executable של התוכנית] ניתן להעביר ב- exec()
  א. full path name [= path name {= string} שמתחיל בתו "/"]
  ב. relative path name

- במקרה של relative path name, הפונקציה exec() 'תמיר' אותו ל- full path name
- איך? על ידי 'שרשור' של ה- relative path name עם string
- איזה string? כזה ששמור במשתנה PATH
  - אחד ממשתני הסביבה של bash
    - 'שומר' רשימה של strings

פקודות פנימיות

- דוגמאות:
  - cd – שינוי ערכו של ה- CWD
  - history
  - set – 'השמת ערך' לאחד ממשתני הסביבה של תכנית ה- bash
  - unset
  - alias

A trivial bash

```
while(1) {
        type_prompt();
        read_command(command);   /* input from terminal */

        parse(command) ;
         /* 'break' the command {= string} into 'tokens'.
            the 1st token is considered to be the command name
            whereas the other tokens are considered as arguments or flags to the command */

        /* now, check if the command name is one that bash can perform.
                for example 'cd' or 'history', etc.
           if yes – carry out the appropriate operations
           if not – create a new process. a child process.
                - the child process is inherits the resources allocated to the parent process,
                        that is to the bash process.
                - including all memory areas [text {code} area, data area, bss area, etc.] allocated to bash
                - open file descriptors
                - other resources
        */
```

A trivial bash (cont.)

```
pid = fork();  /* request to create a child process*/

if(pid < 0) {
        printf("Unable to fork");
        continue;
}

if (pid != 0) {
        /* parent [= bash]  code... */
}
```

A trivial bash (cont.)

```
        else {
                ***execvp(command, params,);  /* child  code ... */***
        }
        /* request 'to return' the allocated resources.
           that is:
              'return' {'disconnect'} all allocated memory areas
              'return' {close} all open file descriptors
              etc.
           and allocate 'new' resources
              text memory area into which to copy the code [machine instructions] in the file 'example'
              data area – for Gx and Gy
              bss area – for arr, and Gz
        …..
        ….


} /* end of while loop */
```